Open Access | https://doi.org/10.25080/wdyr6752

Derivations, Not Just Simulations

Teaching Applied Mathematics with Scientific Python

Michael Zargham[1]

[1]Dynamical Systems Group

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Abstract

Scientific computing in Python is fluent in two of David Tall's three worlds of mathematical learning: NumPy and SciPy handle operational symbolism; matplotlib and reactive notebooks handle conceptual embodiment. The third world, axiomatic formalism — where coordinate transformations, optimality criteria, and invariances live as symbol-manipulation under formal rules — typically stays on paper. SymPy is the SciPy-ecosystem tool that brings it into the same execution environment, and once all three worlds are executable in one notebook the glass-box principle (Buchberger; Drijvers; Artigue) becomes practicable for working scientists.

The worked exemplar is Isaacs's 1951 homicidal-chauffeur differential game. *With this exemplar's parameter settings, naive evader heuristics are caught within 11.3 seconds; saddle-point optimal play survives to 12.1 seconds, but its evader path has a sharp mid-chase kink no physical intuition predicts.* The body derives the optimal feedback symbolically, integrates it backward from the capture circle, and renders the value function as a heat map. The mathematics is canonical; what we claim is the executable implementation: Python modules shipped alongside the article, exercised by an automated test suite.

Keywords mathematics education, symbolic computation, differential games, SymPy, marimo, homicidal chauffeur, glass-box pedagogy, optimal control

1. The problem: teaching advanced applied math to working scientists

Scientific-computing users reach for the SciPy stack when they need numerical methods. This paper argues that the same stack (extended with MyST and a reactive notebook) also operationalises a thirty-year-old theory of how people learn mathematics. We make the case with one worked exemplar: the value function of a 1951 differential game.

The audience we have in mind is the engineer or scientist who reaches for `scipy.integrate.solve_ivp` and `numpy.linalg.eig` reflexively but treats symbolic derivations as something other people do on paper. David Tall [1] named exactly this reader: a *user* of mathematics who needs the mathematical objects working before they need the proofs that justify them. The SciPy stack already serves two of the three worlds Tall describes: *operational-symbolic* procedures (solving, integrating, optimising) and *conceptual-embodiment* (plots, interactive sliders). But the *axiomatic-formal* world, where a symbol *is* the object and an equation is what you manipulate, usually lives in a textbook citation rather than an executable cell.

SymPy closes that gap. The central claim of this paper is that with SymPy in the notebook alongside NumPy/SciPy and matplotlib, the *glass-box principle* [2] (refined by Drijvers and Artigue [3], [4]) finally becomes practicable: the reader can follow the chain from a derived expression, to the function it lambdifies into, to the figure it renders — and verify every

Published Jul 13, 2026

Correspondence to
Michael Zargham
michael.zargham@gmail.com

Open Access

Copyright © 2026 Zargham. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

link, rather than trust one opaque step. SymPy is what unlocks the third world for scientific-computing practice. Proprietary computer-algebra systems (Maple, Mathematica) and other open-source ones (Maxima, Sage) have offered symbolic capabilities for decades; the novel pivot here is having *all three Tall worlds executable in the same open-source Python kernel and notebook*, alongside the NumPy/SciPy/matplotlib idiom the working reader already uses.

We pick the homicidal chauffeur (HC), Isaacs's 1951 differential game, as the worked exemplar because it has every component the argument needs: a clean derivation chain that benefits from symbolic manipulation, an analytical singular-surface structure that makes a striking visualisation, and small enough scope to fit in a 6,000-word notebook. The publication substrate itself (Jupyter, MyST, Curvenote [5], [6], [7]) is part of the demonstration; everything below this paragraph is executable.

Three worlds and three boxes names the two pedagogical frameworks this paper depends on. The case study: the homicidal chauffeur (lens, not subject) introduces the exemplar with the figure that poses the question the rest of the paper answers. Two threads run in parallel through what follows: an *implementation* (the executable derivation chain in `paper.ipynb` plus the modules and tests beside it) and an *argument* (the three-worlds executability category that the implementation grounds). Each load-bears on the other.

2. Three worlds and three boxes

Two pedagogical frameworks structure the rest of the paper. Both come from the mathematics-education research literature, both have roots in the late 1980s, and both translate cleanly into questions about scientific computing.

David Tall [1], [8] describes mathematical cognition as three "worlds" of meaning, each with its own characteristic objects and modes of reasoning. The **conceptual-embodied world** is where mathematics is sensed and pictured: a vector field you can feel the divergence of, a trajectory you can trace with your eye, a slider you can scrub to watch a parameter change behaviour. The **operational-symbolic world** is where mathematics is a procedure you also treat as an object — the process-object duality introduced by A. Sfard [9], elaborated by Tall as the *procept*: $f'(x)$ is both a differentiation procedure you execute on f and an object you can integrate, compose, or compare. The **axiomatic-formal world** is where mathematics is symbol manipulation under formally defined rules: f *is* an element of a function space, f' *is* the linear operator the differentiation rules give you, and a "proof" is a tree of formal rewrites. Each world is a legitimate way of doing mathematics; mature practitioners move between them constantly without naming the moves.

Bruno B. Buchberger [2] gave a complementary principle, articulated at a 1989 computer-algebra colloquium: when teaching a method, learners need a **white-box** phase where they execute every step by hand to understand how the method works, before they can responsibly use a **black-box** tool that wraps the method. M. K. Heid [10] had observed the same pattern empirically: when a computer algebra system (CAS) removes the symbolic-manipulation burden, students still need supervised practice with the structure underneath. Buchberger's principle is *spiral*: today's white-box is tomorrow's black-box, freeing tomorrow's white-box to be something more abstract. Differentiation and integration rules have been in their black-box phase since Heid's 1988 studies, which is what releases this paper's white-box phase to be about Hamiltonian construction and costate derivation rather than chain rules.

Drijvers and Artigue [3], [4] sharpened the Buchberger binary into a third mode: **glass-box**, where the tool is transparent in *derivation* even when the user did not perform every algebraic step by hand. A glass-box workflow lets the reader inspect each intermediate

object the tool produced, ask why each step is justified, and re-run any step with different inputs. The distinction between white-box and glass-box matters for this paper: we do not hand-execute every algebraic step (that would be white-box), but every step we ask SymPy to perform is *inspectable* (glass-box). The glass-box moment: one trajectory through every lens will earn the term with a worked example; the rest of the paper trades on it.

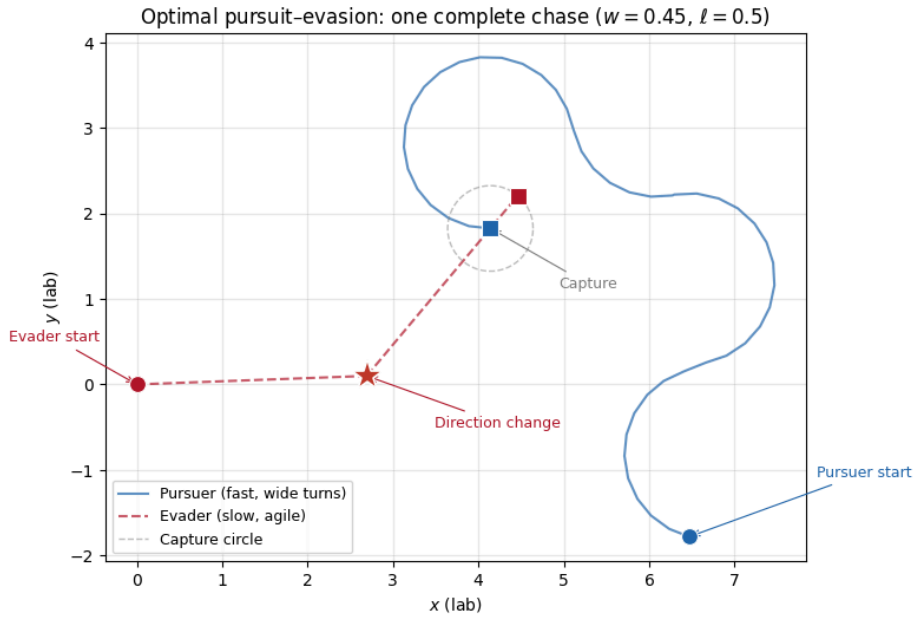
3. The case study: the homicidal chauffeur (lens, not subject)

The homicidal chauffeur (introduced by Rufus Isaacs in a 1951 RAND memorandum [11] and developed in his 1965 textbook [12]) is a two-player differential game. A pursuer moves at constant unit speed but turns under a bounded angular rate (the "chauffeur": fast forward, sluggish to turn); an evader moves at constant speed $w < 1$ (speeds are normalised to the pursuer's, so w is the evader-to-pursuer speed ratio) in any direction it chooses (the "pedestrian": slow, agile). The pursuer wants to capture the evader as quickly as possible by closing within a fixed capture radius ℓ; the evader wants to delay capture as long as possible. Merz [13] gave the complete singular-surface taxonomy (the map of where optimal strategies switch, split, or merge) that makes optimal play tractable; Pachter and Coates [14] is the most-cited modern restatement; Patsko, Botkin, and Kein [15] is the modern history and source for the live demos this paper draws on.

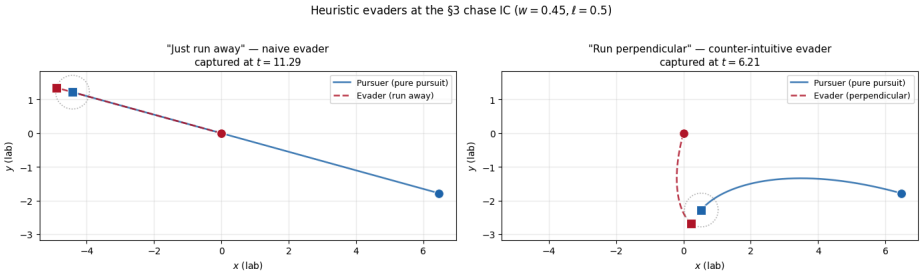
The first figure (`chase_demo`) shows one complete optimal chase. Read it from the ● **circles** (initial positions): the blue pursuer behind and to the right of the red evader at $t = 0$. They evolve forward in time along the solid blue and dashed red curves until the ■ **squares** at the moment of capture. The evader's path has a sharp ⋆, the *dispersal kink*, about halfway through. We use the same visual vocabulary in every figure (all rendered via matplotlib [16]): circles start, squares end, stars mark events; the convention is documented in `plots.py` and walked through inline as each figure appears.

That ⋆ is the question the rest of the paper answers. The second figure (`naive_vs_optimal`) compares two heuristic evaders against the same heuristic pursuer at the same initial condition and parameter settings ($w = 0.45$, $\ell = 0.5$): a "run straight away from the pursuer" evader (left, captured at 11.3 seconds) and a "run perpendicular to the closing direction" evader (right, captured at 6.2 seconds). The saddle-point evader plays differently: captured at 12.1 seconds (now against the optimal pursuer), with the ⋆ kink. The saddle-point theorem says no heuristic evader can do better than the optimal strategy against an optimal pursuer; the 12.1 seconds value is a *bound*, not a behavioural guess. **The question:** where does the ⋆ come from? Why does the evader's optimal path kink at that specific point? This section poses the puzzle; Sections ?? through ?? derive the answer.

```
# §3 motivation figure — lifted from hc-marimo (real chase, dispersal crossing)
from plots import chase_demo
chase_demo()
```




```
# §3 hook — two heuristic evaders at the chase_demo IC.
# See §3 prose for the saddle-point bound interpretation.
from plots import naive_vs_optimal
naive_vs_optimal()
```



4. A CRASH COURSE IN *DIFFERENTIAL GAMES* AND THE *CALCULUS OF VARIATIONS*

Calculus of variations and optimal control are load-bearing tools across applied mathematics: used routinely in aerospace, robotics, control engineering, mathematical biology, economics, and quantitative finance, though rarely taught outside narrow specializations at the undergraduate level. We chose a differential game rather than a single-player optimal-control problem because pursuit-evasion is essentially the children's game of *tag*: every reader has embodied experience of chasing and being chased, intuition to hang the formalism on.

Concretely, the HC state is naturally described in the lab frame with five degrees of freedom (DOF): pursuer position (x_P, y_P) and heading θ, evader position (x_E, y_E). The kinematics are

$$\begin{aligned} \dot{x}_P &= \cos\theta, \qquad \dot{y}_P = \sin\theta, \qquad \dot{\theta} = \phi, \\ \dot{x}_E &= w\cos\psi_{\text{lab}}, \qquad \dot{y}_E = w\sin\psi_{\text{lab}}, \end{aligned} \tag{1}$$

with $|\,\phi\,| \le 1$ the normalised turn-rate control ($v_P = 1$, minimum turning radius $R_{\min} = 1$) and ψ_{lab} the evader's lab-frame heading. Five DOF, two players, two controls. The formulation is well-posed but ungainly: each extra degree of freedom adds a variable that every symbolic derivation downstream must carry.

Two concepts from optimal control are load-bearing for the derivations that follow. We sketch them in one paragraph each.

A **differential game** is the optimal-control generalisation of a saddle-point problem to continuous time. Two players, the pursuer P and the evader E, choose control inputs u_P and u_E (for HC, the turn rate ϕ and the heading ψ_{lab} just defined) that jointly determine the state's evolution $\dot{\boldsymbol{x}} = f(\boldsymbol{x}, u_P, u_E)$. They have opposed objectives (one minimises a running cost $L(\boldsymbol{x}, u_P, u_E)$, the other maximises it), and we look for a *saddle point* (u_P^*, u_E^*) where neither player can improve unilaterally. Isaacs's 1951 contribution was to formulate HC this way and to show that saddle points exist for many adversarial pursuit scenarios; the existence question is the hard part, and the answer depends on the singular-surface structure §?? visits.

The **Pontryagin maximum principle** (PMP) is the calculus-of-variations machinery that locates the saddle point. It introduces a costate vector $\boldsymbol{p}(t)$ (the analogue of momentum in classical mechanics: it measures how sensitive the optimal cost is to each state component) and a Hamiltonian $H(\boldsymbol{x}, \boldsymbol{p}, u_P, u_E) = \boldsymbol{p} \cdot f + L$. The optimal controls minimise (resp. maximise) H over their feasible sets, and the state-costate pair evolves under Hamilton's equations $\dot{\boldsymbol{x}} = \partial H/\partial \boldsymbol{p}$, $\dot{\boldsymbol{p}} = -\partial H/\partial \boldsymbol{x}$. For HC the Hamiltonian turns out to be linear in the pursuer control ϕ, which makes the optimal pursuer control "bang-bang" (always at the boundary of its feasible set, with sign chosen by a *switching function*). This linearity is the structural fact §?? exploits to derive the switching function in one chained SymPy call.

The following notation quickstart names the symbols this paper uses; each one is also defined at first technical use, and the supporting material holds the complete glossary. With these handles in place, §?? turns to the HC-specific work: collapsing the 5-DOF lab-frame setup to the 2-DOF body-frame system every subsequent derivation operates on.

Notation quickstart. Just the abstract objects of a Pontryagin / differential-game setup so the prose has handles to reach for. The HC-specific symbols ($x_1, x_2, \phi, \psi, \sigma, \ldots$) are introduced where they first appear in §??–§??; the full notation reference is in the **supporting material**.

Term	Notation	Quickstart description
State vector	$\boldsymbol{x}$	The system's evolving variables; in HC, the 2-DOF body-frame relative position
Player controls	$u_P,\ u_E$	The two players' choices; in HC, pursuer turn rate ϕ and evader heading ψ
Costate / adjoint	$\boldsymbol{p}$	Pontryagin's costate vector; equals ∇V^*, satisfies $\dot{\boldsymbol{p}} = -\partial H/\partial \boldsymbol{x}$
Hamiltonian	$H(\boldsymbol{x}, \boldsymbol{p}, u_P, u_E)$	The Pontryagin function; its gradient drives the state and costate ordinary differential equations (ODEs)
Value function	$V^*(\boldsymbol{x})$	The saddle-point optimal cost from state $\boldsymbol{x}$; for HC, the optimal capture time
Speed ratio	$w = v_E/v_P \in (0, 1)$	The pursuer is faster in this paper's regime

That's the vocabulary the rest of the paper operationalises in SymPy; each symbol reappears with full context where it is used.

5. Reducing HC to body-frame coordinates (the symbolic-view warm-up)

The *game* does not depend on all five DOF of §??'s lab-frame state. The same game is played by a pursuer at (x_P, y_P) heading north as by a pursuer at the origin heading north: only the *relative* position of the evader matters, and only relative to the pursuer's heading. Isaacs [12, pp. 297-300] gave the reduction: rotate and translate the lab frame so the pursuer sits at the origin facing the body-frame $+x_2$ axis. The evader's heading relabels: $\psi \equiv \psi_{\text{lab}} - \theta$, the angle the evader's velocity makes with the pursuer's heading.

The same 5-DOF lab kinematics, now as a SymPy `Matrix`:

```
# §5 — Lab-frame absolute kinematics: the 5-DOF state
# (x_P, y_P, θ, x_E, y_E) and its time derivatives, normalised so v_P = 1
# and R_min = 1. This is what the body-frame reduction below collapses.
from derivations import absolute_kinematics
sp.Eq(
    sp.Matrix([
        sp.Symbol(r"\dot{x}_P"), sp.Symbol(r"\dot{y}_P"),
        sp.Symbol(r"\dot{\theta}"),
        sp.Symbol(r"\dot{x}_E"), sp.Symbol(r"\dot{y}_E"),
    ]),
    absolute_kinematics(),
)
```

```
⎡ \dot{x}_P  ⎤   ⎡          cos(θ)          ⎤
⎢            ⎥   ⎢                          ⎥
⎢ \dot{y}_P  ⎥   ⎢          sin(θ)          ⎥
⎢            ⎥   ⎢                          ⎥
⎢\dot{\theta}⎥ = ⎢            φ             ⎥
⎢            ⎥   ⎢                          ⎥
⎢ \dot{x}_E  ⎥   ⎢w·cos(\psi_{\mathrm{lab}})⎥
⎢            ⎥   ⎢                          ⎥
⎣ \dot{y}_E  ⎦   ⎣w·sin(\psi_{\mathrm{lab}})⎦
```

Isaacs's rotation+translation written symbolically: the body-frame relative position (x_1, x_2) is a fixed linear combination of the lab-frame relative displacement $(\Delta x, \Delta y)$ rotated by $-\theta$:

```
# §5 — Display the lab → body coordinate substitution (rotation+translation).
from derivations import body_frame_substitution
sp.Eq(
    sp.Matrix([sp.Symbol("x_1"), sp.Symbol("x_2")]),
    body_frame_substitution(),
)
```

```
⎡x₁⎤   ⎡(-x_E + x_P)·sin(θ) + (y_E - y_P)·cos(θ)⎤
⎢  ⎥ = ⎢                                       ⎥
⎣x₂⎦   ⎣(x_E - x_P)·cos(θ) + (y_E - y_P)·sin(θ) ⎦
```

Watch this: these two ingredients, the lab kinematics and the body-frame substitution, are enough for SymPy [17] to derive the reduced body-frame dynamics $(\dot{x}_1, \dot{x}_2) = (f_1, f_2)$ for us. The chain is `sp.diff` (chain-rule the substitution through time), `sp.subs` (drop in the lab kinematics + relabel $\psi_{\text{lab}} = \psi + \theta$), invert the body-frame substitution so the answer reads in (x_1, x_2), and `sp.trigsimp` to collapse the trig. We do it in three small cells so the intermediate forms are visible.

First we set up time-dependent versions of the lab states (so `sp.diff` knows what depends on t) and write $x_1(t)$, $x_2(t)$ as the body-frame combinations:

```
# §5 — Set up SymPy time-functions for the lab states and write
# the body-frame relative position as combinations of those functions.
# We import phi, psi, w, x1, x2 (and the costates p1, p2 needed by §6).
from derivations import phi, psi, w, x1, x2, p1, p2

t = sp.Symbol("t", real=True)
xP_t = sp.Function("x_P")(t)
yP_t = sp.Function("y_P")(t)
theta_t = sp.Function(r"\theta")(t)
xE_t = sp.Function("x_E")(t)
yE_t = sp.Function("y_E")(t)
psi_lab_t = sp.Symbol(r"\psi_{\mathrm{lab}}", real=True)

x1_of_t = -(xE_t - xP_t) * sp.sin(theta_t) + (yE_t - yP_t) * sp.cos(theta_t)
x2_of_t =  (xE_t - xP_t) * sp.cos(theta_t) + (yE_t - yP_t) * sp.sin(theta_t)
sp.Eq(sp.Function(r"x_1")(t), x1_of_t)
```

$$x_1(t) = (-x_E(t) + x_P(t))\sin(\theta(t)) + (y_E(t) - y_P(t))\cos(\theta(t))$$

Now take d/dt of $x_1(t)$. SymPy's chain rule produces a mess of `Derivative(…)` terms — the *unreduced* time-derivative of the body-frame substitution:

```
# §5 — Time-differentiate x_1(t). SymPy chain-rules through the
# substitution and leaves Derivative(...) terms for the lab states.
x1_dot_raw = sp.diff(x1_of_t, t)
sp.Eq(sp.Symbol(r"\dot{x}_1^{\,\mathrm{raw}}"), x1_dot_raw)
```

$$\dot{x}_1^{\mathrm{raw}} = (-x_E(t) + x_P(t))\cos(\theta(t))\frac{d}{dt}\theta(t) - (y_E(t) - y_P(t))\sin(\theta(t))\frac{d}{dt}\theta(t) + \left(-\frac{d}{dt}x_E(t) + \frac{d}{dt}x_P(t)\right)\sin(\theta(t)) + \left(\frac{d}{dt}y_E(t) - \frac{d}{dt}y_P(t)\right)\cos(\theta(t))$$

Substitute the lab kinematics, relabel ψ_{lab}, invert the body-frame substitution so the answer reads in (x_1, x_2), and let `sp.trigsimp` finish the trig identities. The same chain applied to $x_2(t)$ gives f_2.

```
# §5 — Final SymPy chain: subs(lab_kin) → relabel ψ_lab → subs(inv_body) → trigsimp.
# Display as a Python list of Eq (avoids the sp.Matrix-of-Eq deprecation).
lab_kin = {
    sp.diff(xP_t, t): sp.cos(theta_t),
    sp.diff(yP_t, t): sp.sin(theta_t),
    sp.diff(theta_t, t): phi,
    sp.diff(xE_t, t): w * sp.cos(psi_lab_t),
    sp.diff(yE_t, t): w * sp.sin(psi_lab_t),
}
inv_body = {
    xE_t: -x1 * sp.sin(theta_t) + x2 * sp.cos(theta_t) + xP_t,
    yE_t:  x1 * sp.cos(theta_t) + x2 * sp.sin(theta_t) + yP_t,
}

def derive(xi_of_t):
    return sp.trigsimp(sp.expand(
        sp.diff(xi_of_t, t).subs(lab_kin).subs(psi_lab_t, psi + theta_t).subs(inv_body)
    ))

f1, f2 = derive(x1_of_t), derive(x2_of_t)
[sp.Eq(sp.Symbol(r"\dot{x}_1"), f1),
 sp.Eq(sp.Symbol(r"\dot{x}_2"), f2)]
```

$$\left[\dot{x}_1 = -\phi x_2 + w\sin(\psi),\ \ \dot{x}_2 = \phi x_1 + w\cos(\psi) - 1\right]$$

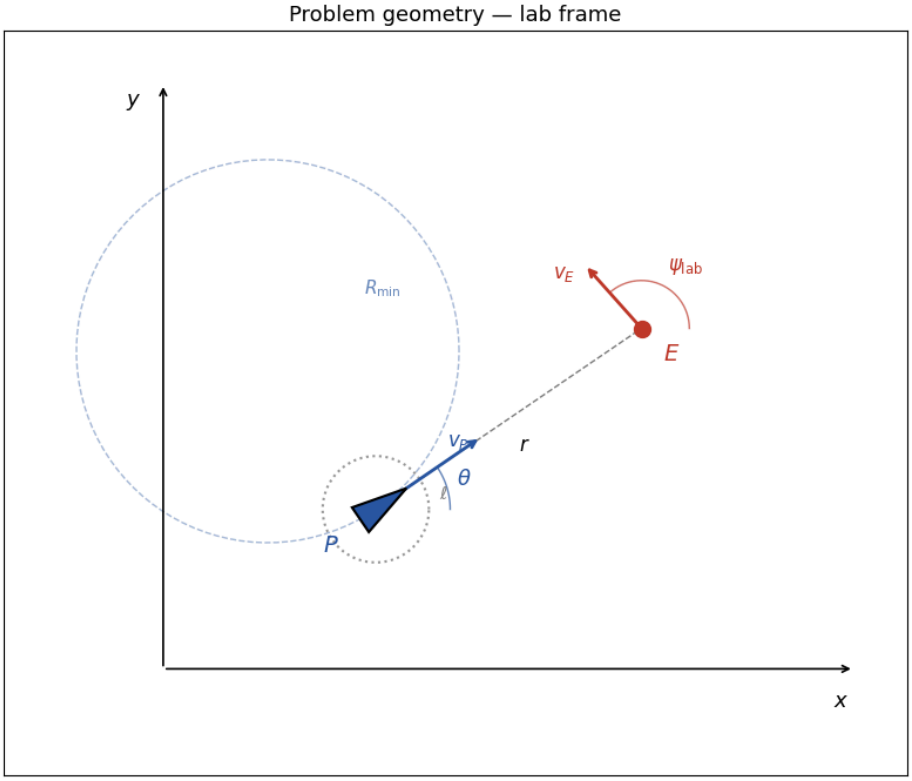
SymPy did the algebra Isaacs would have done by hand. The reader can re-run with a swapped convention (say, evader $\sin \leftrightarrow \cos$) and watch the simplified form change.

Coordinate changes that exploit symmetry to reduce state-space dimension are a standard move in dynamical-systems analysis — and often necessary in differential games, where the full state may be too high-dimensional for a value-function field to be tractable. The reduction is also *invertible* given the pursuer's initial lab pose $(x_P(0), y_P(0), \theta(0))$: composing the body-frame trajectory with the integrated lab-frame pursuer state recovers the full lab-frame chase. That is why §??'s `chase_demo` (lab frame) and §??'s `dispersal_crossing` (body + costate + strategy) render the *same* trajectory from different coordinate views.

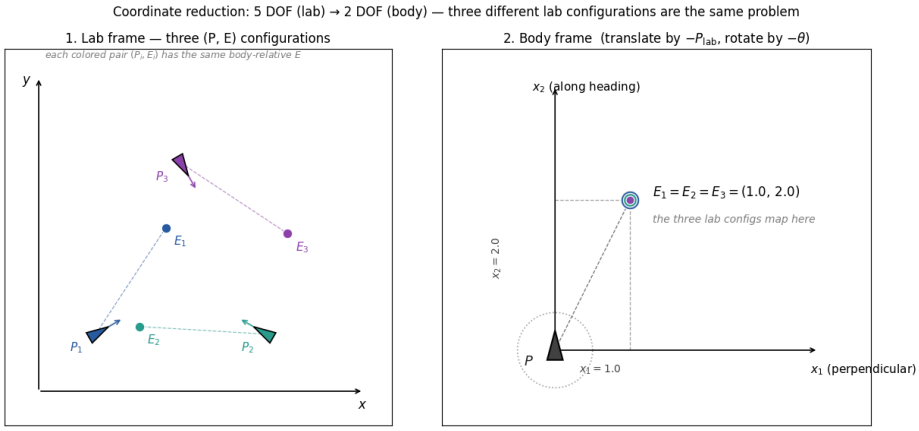
Two figures support the reduction. `problem_geometry` (the lab-frame schematic) names the symbols visually: pursuer heading θ, turning circle of radius $R_{\min}$, capture circle of radius ℓ around the pursuer, evader velocity vector at angle ψ_{lab} to the lab $+x$ axis. `coordinate_progression` (the pair of panels) shows *why* the reduction is interesting: three of the infinitely many lab configurations sharing one relative geometry (pursuer here facing east, there facing northwest, elsewhere facing southeast) all collapse to the *same point* on the body-frame panel. Infinitely many lab states, one game state.

The reduction also changes what the optimal-control problem solves for. In lab coordinates the value function would depend on all five lab DOF; in body coordinates it depends on (x_1, x_2) only. The synthesis: the value function as backward reachable set's `reachable_set_heatmap` renders a 2-D field; without the reduction it would be a 5-D field nobody could visualise. With the body-frame reduction in hand, §?? writes down the Hamiltonian and derives the optimal controls.

```
# §5 supporting figure — lab-frame schematic (problem geometry).
# Sets up the 5-DOF lab state (P_lab, E_lab, θ) the body-frame
# reduction above collapsed to 2 DOF.
from plots import problem_geometry
problem_geometry()
```



```
# §5 figure — three lab configurations collapse to ONE body-frame point.
# Makes the dimensional-reduction insight visible: the body frame is an
# equivalence class on the 5-DOF lab state, not just a relabelling.
from plots import coordinate_progression
coordinate_progression()
```



6. Defining V^* and constructing the Hamiltonian

The **value function** $V^*(\boldsymbol{x})$ is what we are after: the optimal capture time when the game starts from body state $\boldsymbol{x}$ with both players following the saddle-point strategy. The reachable set at time T is its sublevel $R(T) = \{\, \boldsymbol{x} : V^*(\boldsymbol{x}) \le T \,\}$, the set of starting positions from which capture is possible within T seconds of optimal play. V^* exists everywhere the game is well-posed; §?? will render the whole field as a heat map. This section derives the differential equations that connect V^* to the dynamics.

The plan is the *method of characteristics*, run backward: along special curves (the characteristics — here, the optimal trajectories themselves), a partial differential equation (PDE) reduces to ordinary differential equations we integrate directly. The terminal condition is fixed: at the moment of capture, the state lies on the circle $x_1^2 + x_2^2 = \ell^2$ and $V^* = 0$. Optimal trajectories arrive at that boundary; integrating their dynamics backward in time from the boundary recovers $V^*(\boldsymbol{x})$ everywhere the characteristics reach. The Hamilton-Jacobi-Isaacs (HJI) PDE [18] is the axiomatic-formal statement of this: V^* is its *viscosity solution*, the generalised-solution notion for PDEs with corners (a historical term, unrelated to physical viscosity). The synthesis: the value function as backward reachable set makes the anchor explicit; here we set up the characteristics by writing the Hamiltonian.

For the time-optimal differential game the Hamiltonian assembles directly: $H = p_1 f_1 + p_2 f_2 + 1$, where p_1, p_2 are the costate components dual to (x_1, x_2) and the +1 comes from minimising the running cost $L = 1$ (capture time itself). One line of SymPy multiplication:

```
# §6 — Hamiltonian assembly: H = p_1 f_1 + p_2 f_2 + 1 (Pontryagin, L=1).
H = p1 * f1 + p2 * f2 + 1
sp.Eq(sp.Symbol("H"), H)
```

$$H = p_1(-\phi x_2 + w\sin(\psi)) + p_2(\phi x_1 + w\cos(\psi) - 1) + 1$$

Notice that H is *linear* in the pursuer control ϕ: that is the structural observation the next cell exploits. One SymPy call (`sp.expand(H).coeff(phi)`) extracts the coefficient of ϕ; the **switching function** $\sigma = p_2 x_1 - p_1 x_2$ falls out directly:

```
# §6 — Switching function via one SymPy call: σ = sp.expand(H).coeff(φ).
sigma = sp.expand(H).coeff(phi)
sp.Eq(sp.Symbol(r"\sigma"), sigma)
```

$$\sigma = -p_1 x_2 + p_2 x_1$$

PMP's bang-bang minimiser is then $\phi^* = -\mathrm{sign}(\sigma)$, because minimising a linear function over $[-1, 1]$ always lands on the boundary opposite the slope's sign [13, §3.2]. The locus $\sigma = 0$ is where the bang-bang control switches; in HC this locus is a curve in body space, the *dispersal surface* §?? visits.

```
# §6 — Bang-bang pursuer control: φ* = -sign(σ).
phi_star = -sp.sign(sigma)
sp.Eq(sp.Symbol(r"\phi^*"), phi_star)
```

$$\phi^* = -\mathrm{sign}\left(-p_1 x_2 + p_2 x_1\right)$$

The evader's optimal heading falls out the same way: $\psi^* = \mathrm{atan2}(p_1, p_2)$ maximises the evader's contribution to H, which works out to the evader pointing its velocity vector along the costate $\boldsymbol{p}$.

Geometric reading: the costate is the gradient of the value function, and the evader runs straight up that gradient. This is the single most useful intuition the rest of the paper exploits.

```
# §6 — Optimal evader heading: ψ* = atan2(p_1, p_2).
psi_star = sp.atan2(p1, p2)
sp.Eq(sp.Symbol(r"\psi^*"), psi_star)
```

$$\psi^* = \mathrm{atan}_2\left(p_1, p_2\right)$$

One technical detail anchors §??'s backward integration: the usable part of the terminal capture circle is the arc where $\sin\alpha > w$ [13, §4]; only on that arc can the pursuer prevent the evader from escaping at the moment of contact. Backward characteristics start from this arc, parameterised by α, the angular position on the capture circle where each characteristic terminates. The synthesis: the value function as backward reachable set's `value_function_grid` sweeps α and integrates each characteristic backward to build the field.

The glass-box moment: one trajectory through every lens takes one optimal play through every coordinate lens at once. The cells we just built (Hamiltonian, switching function, bang-bang control, evader heading) are the analytical content the next figure visualises.

7. The glass-box moment: one trajectory through every lens

Substituting ϕ^* and ψ^* into the body-frame dynamics gives a 4-dimensional autonomous ODE in (x_1, x_2, p_1, p_2). The next cell displays it: the state evolves under the closed-loop dynamics with the bang-bang control $\phi^* = -\mathrm{sign}(\sigma)$ and the evader-velocity components

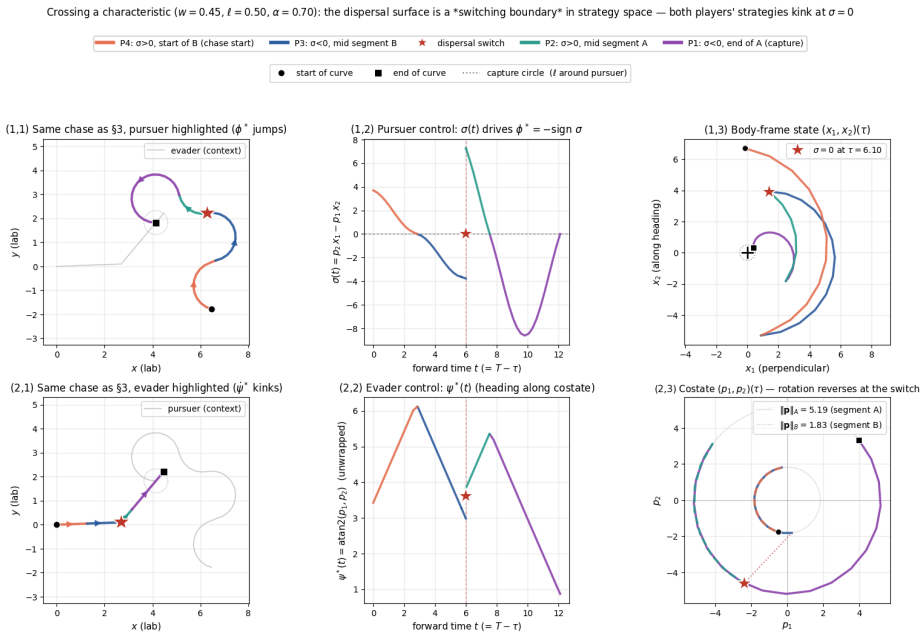
determined by $\boldsymbol{p}$. This 4-D system is the *axiomatic-formal object* the rest of §?? visualises: every figure below is a slice through one solution of this ODE.

```
# §7 — The 4-D characteristic ODE with φ*, ψ* substituted (the autonomous closed-loop system).
from derivations import characteristic_system
sp.Eq(
    sp.Matrix([
        sp.Symbol(r"\dot{x}_1"), sp.Symbol(r"\dot{x}_2"),
        sp.Symbol(r"\dot{p}_1"), sp.Symbol(r"\dot{p}_2"),
    ]),
    sp.Matrix(list(characteristic_system())),
)
```

$$
\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{p}_1 \\ \dot{p}_2 \end{bmatrix}
=
\begin{bmatrix}
\dfrac{p_1 \cdot w}{\sqrt{p_1^{2} + p_2^{2}}} + x_2 \cdot \operatorname{sign}(-p_1 \cdot x_2 + p_2 \cdot x_1) \\
\dfrac{p_2 \cdot w}{\sqrt{p_1^{2} + p_2^{2}}} - x_1 \cdot \operatorname{sign}(-p_1 \cdot x_2 + p_2 \cdot x_1) - 1 \\
p_2 \cdot \operatorname{sign}(-p_1 \cdot x_2 + p_2 \cdot x_1) \\
-p_1 \cdot \operatorname{sign}(-p_1 \cdot x_2 + p_2 \cdot x_1)
\end{bmatrix}
$$

The 6-panel `dispersal_crossing` figure shows one optimal chase through six coordinate views simultaneously. The chase is the same one §??'s `chase_demo` motivates; here it is decomposed into its analytical structure. We take the panels in turn:

```
# §7 GLASS-BOX EXEMPLAR — one optimal play, six coordinate views.
# Panel layout + reading guide in the following §7 prose.
from plots import dispersal_crossing
dispersal_crossing()
```



(1,1) Lab frame, pursuer highlighted. The pursuer's trajectory traces an S-shape, with the bang-bang ϕ^* jumping at the dispersal switch ($\star$). The evader's path is greyed.

(1,2) Strategy: $\sigma(t)$. The switching function plotted forward in time. The curve changes sign at the $\star$ and at two smooth crossings either side of it; each sign change starts a new colour segment in (1,1). The $\star$ marks the dispersal switch: at $\sigma = 0$ the pursuer's optimal turning direction is undetermined, and the trajectory bifurcates into two regimes; the locus of such points is a *dispersal surface*, a boundary where two distinct optimal paths become available at once [13], [19].

(1,3) State: body-frame trajectory. The same chase in body coordinates (x_1, x_2). The evader spirals inward toward the capture circle; the spiral pitch flips at the dispersal crossing. This panel is the one body-frame picture that stands in for every lab-frame chase sharing the same relative geometry, including (1,1) and (2,1).

(2,1) Lab frame, evader highlighted. Same chase, opposite highlight. The evader's path is the smooth-then-kinked curve we previewed in §??'s `chase_demo`. The pursuer is greyed; the $\star$ is still there.

(2,2) Strategy: $\psi^*(t)$. The evader's optimal heading, forward in time. Like ϕ^*, ψ^* jumps at the $\star$; the heading-angle discontinuity is what produces the visible kink in the evader's lab-frame path.

(2,3) Costate: (p_1, p_2) phase portrait. The costate trajectory. Two concentric circles of radii $|\,\boldsymbol{p}\,|_B$ and $|\,\boldsymbol{p}\,|_A$ (the costate magnitudes before and after the dispersal) are connected by a dotted red chord at the switch: V^*'s gradient is multi-valued at the dispersal surface, and the costate magnitude jumps pre- and post-switch.

The chase rendered across all six panels is recovered from `chase_demo`'s hardcoded arrays by stitching two optimal characteristics joined at the dispersal point ($\alpha_A = 40^\circ$ post-switch, $\alpha_B = 95^\circ$ pre-switch), so the chase in §?? and the analytical decomposition in §?? are the *same* chase. The plumbing lives in `plots._recover_chase_from_demo`; the costate norms differ across the switch ($|\,\boldsymbol{p}\,|_B \approx 1.83$, $|\,\boldsymbol{p}\,|_A \approx 5.18$), which is exactly what makes the dispersal surface a boundary in strategy space.

This is the glass-box moment. Every panel of `dispersal_crossing` renders a slice of the *same* 4-D ODE solution. The transformations between coordinate views (lab $\leftrightarrow$ body $\leftrightarrow$ strategy $\leftrightarrow$ costate) are now executable cells, not paragraph headers. The reader can re-run with different (w, ℓ), re-stitch with different (α_A, α_B), or call any of `derivations.py`, `numerics.py`, `plots.py` directly from a fresh notebook to inspect any intermediate object. If one optimal chase has structure this rich, what does the whole space of starts look like? The synthesis: the value function as backward reachable set answers — and earns the §?? promise of *verifying the whole chain* via the costate ODE, conservation lemma, and lambdified right-hand side (RHS) that the heat map sits on top of; the supporting material runs the live symbolic-versus-numerical cross-check.

8. The synthesis: the value function as backward reachable set

This section has two goals: (1) render $V^*(x_1, x_2)$ over the body frame as a heat map, and (2) finish the analytical machinery — costate ODE, conservation invariant, and the SymPy $\rightarrow$ NumPy seam (where symbolic expressions compile into fast numeric callables) — that the heat-map cell depends on. We derive the costate dynamics first, then render and interpret the synthesis figure, then finish the conservation and seam machinery.

The **costate (adjoint) ODE** comes from Pontryagin's principle: $\dot{p}_i = -\partial H / \partial x_i$. Two `sp.diff` calls:

```
# §8 — Costate (adjoint) ODE for p_1: dp_1/dt = -∂H/∂x_1.
# Pontryagin's maximum principle, written as a one-line SymPy call.
sp.Eq(sp.Symbol(r"\dot{p}_1"), -sp.diff(H, x1))
```

$$\dot{p}_1 = -p_2\phi$$

```
# §8 — Costate ODE for p_2: dp_2/dt = -∂H/∂x_2.
sp.Eq(sp.Symbol(r"\dot{p}_2"), -sp.diff(H, x2))
```

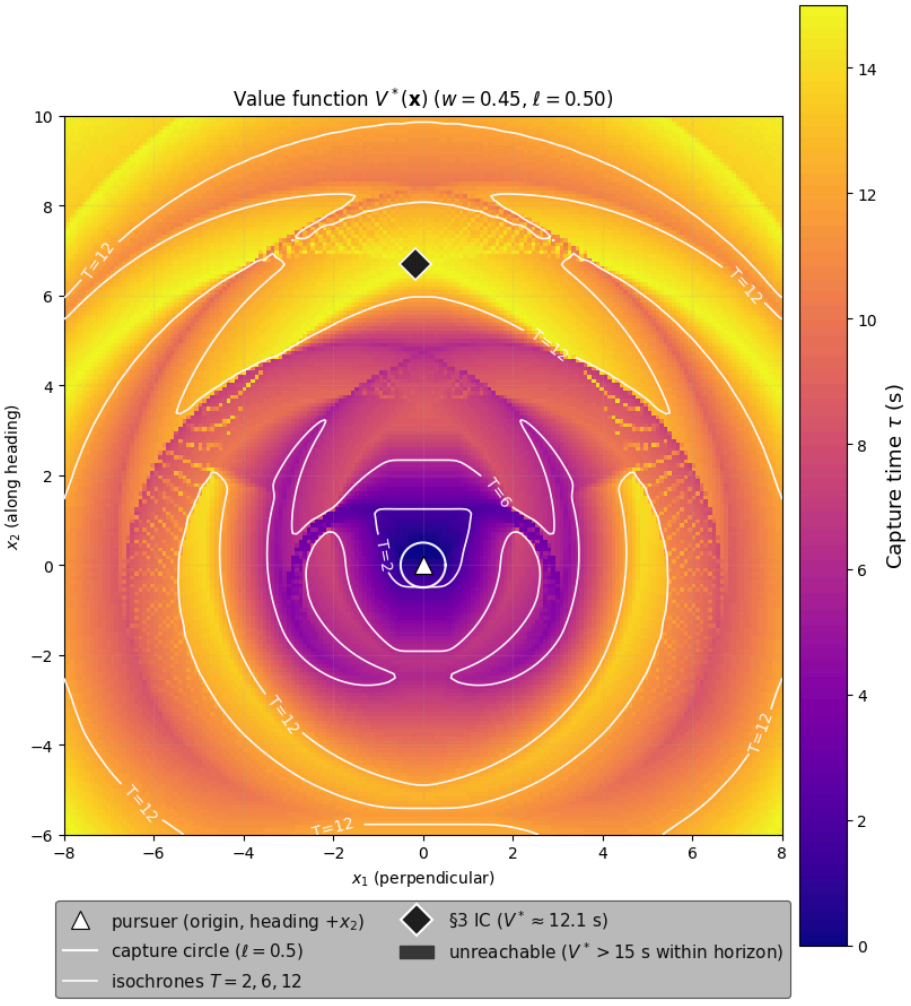
$$\dot{p}_2 = p_1\phi$$

With the adjoint dynamics in hand, we sweep 600 backward characteristics from the usable terminal arc, integrate each one back to time-to-go $\tau = 15$ s, and interpolate the (x_1, x_2, τ) samples onto a 180×180 grid. Plasma colour codes capture time from 0 (purple, on the capture circle) to 15 s (yellow at the cap). The pursuer triangle sits at the origin with heading $+x_2$; white isochrones at $T = 2, 6, 12$ trace level sets of V^*; the capture circle itself is, by definition, the $T = 0$ isochrone.

```
# §8 SYNTHESIS — V*(x) on a body-frame grid; see §8 prose for the
# 15-s cap, IC anchor, and barrier-surface interpretations.
# (Bump _VFG_CACHE_VERSION in numerics.py to invalidate the .npz cache.)
from numerics import value_function_grid
from plots import reachable_set_heatmap

W_REF, ELL_REF = 0.45, 0.5
ISOCHRONE_LEVELS = (2.0, 6.0, 12.0)

x1_grid, x2_grid, V_grid, reachable_mask, isochrones = value_function_grid(
    W_REF, ELL_REF,
    T_max_cap=15.0,
    grid_extent=(-8.0, 8.0, -6.0, 10.0),
    n_grid=180,
    n_traj=600,
    n_time_samples=400,
    isochrone_levels=ISOCHRONE_LEVELS,
)
reachable_set_heatmap(
    x1_grid, x2_grid, V_grid, reachable_mask, W_REF, ELL_REF,
    T_max_cap=15.0, vmax_color=15.0,
    isochrone_levels=ISOCHRONE_LEVELS,
    isochrone_curves=isochrones,
    highlight_ic=(-0.166, 6.708, "§3 IC"),
    highlight_ic_value=12.1,
)
```



The black diamond is the §?? `chase_demo` initial condition, mapped through the body-frame substitution to $(-0.166, 6.708)$. The legend's $V^*(\diamond) \approx 12.1$ s is plumbed in directly from `chase_demo`'s measured saddle-point capture time; the heat map's interpolation reads nearly three seconds high here, because the backward fan does not pass exactly through this point and linear interpolation blends in samples from neighbouring, longer-time characteristics. The same chase thus appears in §?? and §?? with the same capture time.

The **15-second cap is a *rendering* horizon, not a game-theoretic claim**. These parameters sit in the well-behaved region of the game: Isaacs's capture criterion [12] at $\ell = 0.5$, $R_{\min} = 1$ puts the escape threshold near $w \approx 0.95$, so at $w = 0.45$ the pursuer catches from every initial condition. Cells the angular fan does not reach within 15 seconds would render in dark grey (the legend's "unreachable" entry); none occur within this figure's extent. Genuinely no-capture regimes, and stranger singular-surface structures, exist elsewhere in the (w, ℓ) parameter plane [13], [18, §3].

The sharp boundary where the dark inner kidney (the region around the pursuer, roughly $|\, x_1 \,| \le 3$, $-2 \le x_2 \le 4$) meets the lighter outer field is the *barrier singular surface*: V^*'s level sets meet at a corner there, with V^* jumping from $\sim$2 s to $\sim$6 s over less than 0.5 body-frame units. Sampling-density checks (`n_traj` $\in \{100, 300, 600\}$) confirm it as a real HC feature, not a sampling artifact.

The remaining machinery follows. The **conservation lemma** shows that $d/dt \,|\, \boldsymbol{p} \,|^2 = 0$ along every optimal characteristic. The chain-rule sum is held un-evaluated with `sp.Add(..., evaluate=False)` so SymPy's auto-simplifier doesn't collapse it before the reader sees the structure; the next cell's `sp.simplify` is what actually drives the result to literal zero:

```
# §8 — Conservation lemma 1: chain-rule sum, kept unevaluated
# (sp.Add(..., evaluate=False)) so sp.simplify drives the cancellation in lemma 2.
p1_dot = -sp.diff(H, x1)
p2_dot = -sp.diff(H, x2)
drift_raw = sp.Add(2 * p1 * p1_dot, 2 * p2 * p2_dot, evaluate=False)
sp.Eq(sp.Symbol(r"\tfrac{d}{dt}\|\mathbf{p}\|^2_{\,\text{raw}}"), drift_raw)
```

```
\tfrac{d}{dt}\|\mathbf{p}\|_{\,\text{raw}}__2 = -2·p₁·p₂·φ + 2·p₁·p₂·φ
```

```
# §8 — Conservation lemma 2: sp.simplify drives the drift to literal zero.
drift = sp.simplify(drift_raw)
sp.Eq(sp.Symbol(r"\tfrac{d}{dt}\|\mathbf{p}\|^2"), drift)
```

```
\tfrac{d}{dt}\|\mathbf{p}\|² = 0
```

Watch this: SymPy did in one line the cancellation Isaacs (1965) verified by hand. The conservation invariant is now executable evidence rather than a textbook claim.

The seam. `numerics.lambdify_rhs` calls `sp.lambdify` on the characteristic ODE system, returning a NumPy-callable `rhs(t, state, w)` where `state = [x_1, x_2, p_1, p_2]` and the return value is the 4-vector $[\dot{x}_1, \dot{x}_2, \dot{p}_1, \dot{p}_2]$. This is exactly the signature `scipy.integrate.solve_ivp` consumes when integrating the 600 backward characteristics for the heat map. White-box on the symbolic side (the SymPy expression is inspectable with `sp.srepr`), black-box on the numeric side (a fast callable for the integrator), glass-box because both sides live in the source [17], [20].

To make the seam concrete: at probe state $(x_1, x_2, p_1, p_2) = (1.0, 0.5, 0.3, 0.8)$ with $w = 0.3$, the closed-form derivations give $\sigma = p_2 x_1 - p_1 x_2 = 0.65 > 0$, so $\phi^* = -1$. Then $\dot{x}_1 = -\phi^* x_2 +$

$w \sin\psi^* \approx 0.605$, $\dot{x}_2 = \phi^* x_1 + w\cos\psi^* - 1 \approx -1.719$, $\dot{p}_1 = -p_2\phi^* = 0.8$, $\dot{p}_2 = p_1\phi^* = -0.3$. The lambdified callable returns the same four numbers:

```
# §8 — THE SEAM: SymPy → NumPy via sp.lambdify. Probe-state spot-check.
# Closed-form expected values: see §8 prose immediately above.
from numerics import lambdify_rhs
rhs = lambdify_rhs()
rhs(0.0, [1.0, 0.5, 0.3, 0.8], 0.3)
```

```
[0.605337032476517, −1.71910124672929, 0.8, −0.3]
```

One sentence anchors V^* to its axiomatic-formal definition: it is the unique viscosity solution of the HJI PDE with $V = 0$ on the capture circle [18]. We render V^* via backward characteristics rather than by solving the PDE because shipping a viscosity-solution finite-element solver in 6,000 words is beyond scope; the axiomatic-formal anchor is the PDE itself, cited rather than enacted. Two further figures (`plots.optimal_vector_field` for the bang-bang feedback policy, `plots.trajectory_fan` for the family of optimal characteristics) live in the codebase and ship with the public hc-marimo notebook [16], [21], [22].

9. What just happened: SymPy is the missing world; the SciPy stack now covers all three

Standard SciPy practice covers two of Tall's three worlds. NumPy and SciPy operationalise the **operational-symbolic** world: procedures that solve, integrate, optimise [20], [22]. Matplotlib operationalises the **conceptual-embodied** world: vector fields, trajectories, heat maps you read with your eyes [16]. The **axiomatic-formal** world, symbol-as-object manipulation under rewrite rules, usually lives on paper or in a textbook citation. SymPy [17] is what moves it into the same execution environment.

What changed in this paper because SymPy was in the notebook? The body-frame reduction in §?? ran as a SymPy chain (`sp.diff` + `sp.subs` + `sp.trigsimp`) rather than three pages of indices. The Hamiltonian in §?? assembled in a single line, $H = p_1 f_1 + p_2 f_2 + 1$. The switching function fell out as `sp.expand(H).coeff(phi)` instead of by-hand collection of terms. The costate ODE was a `sp.diff(H, x_i)` call; the conservation lemma a `sp.simplify(...)` that returned literal zero. Without SymPy, the reader's only audit path would be to read Isaacs (1965, pp. 297-320); with SymPy, the reader watches the derivation happen and can re-run any cell with different parameters to verify the chain end-to-end.

The Buchberger principle [2] governs which mode of work belongs where. Differentiation and integration *rules* belong in the black-box mode by now: their white-box phase dates to Heid [10]'s 1988 calculator-classroom studies, and we trust `sp.diff` to apply them. The white-box mode in this paper is therefore *one rung up the recursion*: Hamiltonian construction, costate derivation, switching-function recognition. Drijvers and Artigue [3], [4] sharpened Buchberger's binary into the third, *glass-box*, mode (§??). SymPy is glass-box in this sense: `sp.expand(H).coeff(phi)` is a single call, but the reader can inspect H before and after, ask for `H.atoms()` or `H.free_symbols`, and trace exactly which rule was applied.

One cell in this section illustrates the *limits* of what computer algebra automates. Substituting $\phi^* = -\mathrm{sign}(\sigma)$ into H yields a `Piecewise` expression branching on the sign of σ. The auto-simplifier does not reduce $-\mathrm{sign}(\sigma)\cdot\sigma$ to $-|\,\sigma\,|$ because the recognition step needs a pattern rewrite SymPy does not apply by default. Glass-box does not mean *omniscient*; it means *inspectable*. SymPy executes rewrite tactics; choosing which tactic to invoke still takes creativity the first time. Once made, that choice persists as version-controlled, inspectable, verifiable SymPy code — exactly what `derivations.py` preserves here.

```
# §9 — Live H.subs(φ, φ*). SymPy returns Piecewise — the auto-simplifier
# does not auto-reduce -sign(σ)·σ to -|σ|. See §9 prose for the honest moment.
H_after_phi = H.subs(phi, phi_star)
H_after_phi
```

$$p_1(w\sin(\psi) + x_2\operatorname{sign}(-p_1x_2 + p_2x_1)) + p_2(w\cos(\psi) - x_1\operatorname{sign}(-p_1x_2 + p_2x_1) - 1) + 1$$

The clean closed form, written by Isaacs (1965, p. 304) as $H^* = w \mid \boldsymbol{p} \mid - \mid \sigma \mid - p_2 + 1$, is the saddle-point Hamiltonian the bang-bang substitution *should* reduce to. We display the clean form via the helper in `derivations.py`, then substitute the definitions of $\mid \boldsymbol{p} \mid$ and σ — asserting the equivalence with a one-line `sp.simplify` — to recover Isaacs's typography exactly:

```
# §9 — Closed-form H* (Isaacs 1965, p. 304), via derivations.hamiltonian_saddle().
from derivations import hamiltonian_saddle
sp.Eq(sp.Symbol(r"H^*"), hamiltonian_saddle())
```

$$H^* = -p_2 + w\sqrt{p_1^2 + p_2^2} - |p_1x_2 - p_2x_1| + 1$$

```
# §9 — Same H*, in Isaacs's own symbols. The assert substitutes the
# definitions ‖p‖ = sqrt(p_1² + p_2²) and σ = p_2·x_1 − p_1·x_2 into the
# named-symbol form and verifies it against hamiltonian_saddle(); the
# final line renders Isaacs's (1965, p. 304) typography term for term.
from derivations import hamiltonian_saddle, x1, x2, p1, p2, w
p_norm = sp.Symbol(r"\|\mathbf{p}\|", positive=True)
sigma_sym = sp.Symbol(r"\sigma", real=True)
H_isaacs = w * p_norm - sp.Abs(sigma_sym) - p2 + 1
assert sp.simplify(
    H_isaacs.subs({p_norm: sp.sqrt(p1**2 + p2**2),
                   sigma_sym: p2 * x1 - p1 * x2}) - hamiltonian_saddle()) == 0
sp.Eq(sp.Symbol("H^*"), sp.Symbol(r"w\|\mathbf{p}\| - \left|\sigma\right| - p_2 + 1"))
```

$$H^* = w\|\mathbf{p}\| - |\sigma| - p_2 + 1$$

Sections ?? through ?? are what earn the §?? thesis: **the glass-box principle becomes practicable when all three of Tall's worlds live in the same execution environment.** Standard SciPy practice already keeps the operational-symbolic and embodied worlds in the same notebook; SymPy bridges the axiomatic-formal world onto the same page as the figures it justifies. The seams between worlds (`derivations.py` → `numerics.lambdify_rhs` → `scipy.integrate.solve_ivp` → `matplotlib.Figure`) are what the reader can audit. Without SymPy, the chain has one black-box step (the formal derivation in a citation); with SymPy, every step is glass-box. Conclusion: critical infrastructure + reproducibility + methodology generalises names the publication infrastructure that makes all three worlds live in one artifact.

10. Conclusion: critical infrastructure + reproducibility + methodology generalises

We named four tool families to make this paper work: SymPy for the axiomatic-formal world, NumPy and SciPy for operational-symbolic, matplotlib for embodied, and MyST + Jupyter + Curvenote for the integrating substrate. We credit them all together because the contribution is about the *category* of tools they exemplify, not about any specific stack.

A dynamic computational-publication environment combines text, parameters, symbolic derivations, numerical simulations, and visual elements *in the same artifact*; this paper is one instance of the category, and the public hc-marimo notebook this paper is built on [21], running on the marimo framework [23], is another. The two instances run on different infrastructure (marimo + GitHub Pages versus Jupyter + MyST + Curvenote [5], [6], [7]) and the choice between them is situational, matched to use case and audience; the load-bearing claim is the category itself. Kadie's 2025 SciPy paper on solvable and unsolvable equations in SymPy [24] is another notebook-primary instance in the same proceedings; the architectural choice is no longer unprecedented.

The methodology generalises. The same treatment (symbolic derivation in SymPy, numerical integration in SciPy, visual decomposition in matplotlib, with the seams visible) applies to other control-systems concepts where the formalism is hard to materialise and the embodied view is load-bearing. Three named instances live at hc-marimo: (1) the singular-surface taxonomy applied to autonomous-vehicle (AV) collision avoidance, where dispersal and equivocal surfaces classify decision boundaries [13], [19]; (2) multi-pursuer differential games applied to unmanned-aerial-vehicle (UAV) swarm defense, where coalition value functions extend Isaacs's pursuit framework [25]; (3) forward physical-coordinate chase applied to hardware-in-the-loop (HIL) testing of pursuit controllers: the same characteristic-integration backbone (built from the modern HC restatement [14]) drives controller validation in a simulator. Each ships alongside the figures cut from this paper for space (`plots.optimal_vector_field`, `plots.trajectory_fan`, `plots.conservation_diagnostics`).

Motivation. We have found Tall's three-worlds model, the glass-box principle, and reactive notebooks indispensable communication aids for advanced engineering mathematics across several artifacts at the Dynamical Systems Group: the public hc-marimo notebook this paper is built on, a spacecraft attitude-control lifecycle requirements-traceability demo [26], and an evolution-of-trust showcase built on a compositional game-theory ecosystem [27]. The same three-worlds pedagogy extends to settings where the state space is a data structure rather than a real-valued field and differentiability cannot be assumed [28], [29]; the symbolic substrate need not be SymPy: semantic-web tools such as the Web Ontology Language (OWL) [30] also provide formal inference.

Sharpened pitch. The embodied and interactive modes are particularly load-bearing at the graduate level, where the formalism is hardest to materialise on the page. The axiomatic-formal world being *operationally available* via SymPy (not hand-waved through "see the textbook") unlocks pedagogical ground that pure-numerical SciPy practice cannot reach. Glass-box is the practical mode for that ground: the reader audits the symbolic derivation, watches the lambdified function get integrated, reads the resulting figure, and can ground-truth any step against the next.

Reproducibility. The suite under `tests/` runs in seconds via `uv run pytest tests/` and certifies every body-section invariant the paper relies on. The **supporting material** carries the full audit: the per-test catalog, a live pytest run executed in-notebook, and the symbolic-vs-numerical cross-check on the costate-norm conservation lemma — the same $d/dt \mid \boldsymbol{p} \mid^2 = 0$ that §?? proves by `sp.simplify` is verified numerically to the integrator's noise floor along every characteristic.

The publication substrate makes the integration possible. Crediting the SymPy, NumPy, SciPy, matplotlib, Jupyter, MyST, Curvenote, and marimo teams is therefore not a courtesy line; it is the load-bearing acknowledgement that scientific computing now has the infrastructure to operationalise Tall's three-worlds theory in a single executable artifact. Teaching advanced applied mathematics finally has the substrate it always needed.

Acknowledgments

Tools used: Claude Code (Anthropic), `claude-opus-4-7` (1M context window). The author specified this paper as point-for-point outlines, named all software components with test-driven development up front, and hand-tuned the code, figures, and text throughout the published content. The build order was outline, then code, then detailed outline, then prose. AI assistance produced drafts inside that specification; the author corrected each pass and verified the mathematics against the cited primary sources [12], [13], [14], [18]. The work was conducted in a variation of the purpose-built AI-assisted-authorship harness described and peer-reviewed in [31], whose deterministic test gates (here, 36 pytest assertions and `utils.validate_citations`) and named-human approver field are the verification mechanism this paper inherits. The author takes full responsibility for the accuracy and integrity of the content.

References

[1] D. O. Tall, "The Transition to Formal Thinking in Mathematics," *Mathematics Education Research Journal*, vol. 20, no. 2, pp. 5–24, 2008, [Online]. Available: https://files.eric.ed.gov/fulltext/EJ820231.pdf

[2] B. Buchberger, "Should Students Learn Integration Rules?," *ACM SIGSAM Bulletin*, vol. 24, no. 1, pp. 10–17, 1990.

[3] M. Artigue, "Learning Mathematics in a CAS Environment: The Genesis of a Reflection About Instrumentation and the Dialectics Between Technical and Conceptual Work," *International Journal of Computers for Mathematical Learning*, vol. 7, no. 3, pp. 245–274, 2002, doi: 10.1023/A:1022103903080.

[4] D. O. Tall and E. Dubinsky, "Advanced Mathematical Thinking and the Computer," in *Advanced Mathematical Thinking*, vol. 11, D. O. Tall, Ed., in Mathematics Education Library, vol. 11., Dordrecht: Kluwer Academic Publishers, 1991, pp. 231–248.

[5] T. Kluyver *et al.*, "Jupyter Notebooks — a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, F. Loizides and B. Schmidt, Eds., 2016, pp. 87–90. doi: 10.3233/978-1-61499-649-1-87.

[6] Project Jupyter *et al.*, "Jupyter Book 2 and the MyST Document Stack: A modular, extensible, web-native stack for authoring and publishing computational narratives," in *Proceedings of the 24th Python in Science Conference*, in Proceedings of the Python in Science Conference. 2025, pp. 173–193. doi: 10.25080/hwcj9957.

[7] Curvenote Inc., "Curvenote: a continuous-publishing platform for scientific content." [Online]. Available: https://curvenote.com/

[8] D. O. Tall, *How Humans Learn to Think Mathematically: Exploring the Three Worlds of Mathematics*. in Learning in Doing: Social, Cognitive and Computational Perspectives. Cambridge: Cambridge University Press, 2013.

[9] A. Sfard, "On the Dual Nature of Mathematical Conceptions: Reflections on Processes and Objects as Different Sides of the Same Coin," *Educational Studies in Mathematics*, vol. 22, no. 1, pp. 1–36, 1991, doi: 10.1007/BF00302715.

[10] M. K. Heid, "Resequencing Skills and Concepts in Applied Calculus Using the Computer as a Tool," *Journal for Research in Mathematics Education*, vol. 19, no. 1, pp. 3–25, 1988, [Online]. Available: https://www.jstor.org/stable/749108

[11] R. Isaacs, "Games of Pursuit," techreport P-257, 1951.

[12] R. Isaacs, *Differential Games: A Mathematical Theory with Applications to Warfare and Pursuit, Control and Optimization*. New York: John Wiley & Sons, 1965.

[13] A. W. Merz, "The Homicidal Chauffeur — A Differential Game." 1971.

[14] M. Pachter and S. Coates, "The Classical Homicidal Chauffeur Game," *Dynamic Games and Applications*, vol. 9, no. 3, pp. 800–850, 2019, doi: 10.1007/s13235-018-0264-8.

[15] V. S. Patsko and V. L. Turova, "Homicidal Chauffeur Game: History and Modern Studies," in *Advances in Dynamic Games: Theory, Applications, and Numerical Methods for Differential and Stochastic Games*, vol. 11, M. Breton and K. Szajowski, Eds., in Annals of the International Society of Dynamic Games, vol. 11., Birkhäuser Boston, 2011, pp. 227–251. doi: 10.1007/978-0-8176-8089-3_12.

[16] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.

[17] A. Meurer *et al.*, "SymPy: symbolic computing in Python," *PeerJ Computer Science*, vol. 3, p. e103, 2017, doi: 10.7717/peerj-cs.103.

[18] M. Bardi, M. Falcone, and P. Soravia, "Numerical Methods for Pursuit-Evasion Games via Viscosity Solutions," in *Stochastic and Differential Games: Theory and Numerical Methods*, vol. 4, M. Bardi, T. E. S. Raghavan, and T. Parthasarathy, Eds., in Annals of the International Society of Dynamic Games, vol. 4., Boston: Birkhäuser, 1999, pp. 105–175. doi: 10.1007/978-1-4612-1592-9_3.

[19] P. Bernhard, "Singular Surfaces in Differential Games: An Introduction," in *Differential Games and Applications*, vol. 3, P. Hagedorn, H. W. Knobloch, and G. J. Olsder, Eds., in Lecture Notes in Control and Information Sciences, vol. 3., Springer, 1977, pp. 1–33.

[20] P. Virtanen *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[21] M. Zargham, "hc-marimo: source materials for SciPy 2026 — mathematics education with scientific Python." [Online]. Available: https://mzargham.github.io/hc-marimo/

[22] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[23] A. Agrawal and M. Scolnick, "marimo: an open-source reactive notebook for Python." Zenodo, 2023. doi: 10.5281/zenodo.12735329.

[24] C. Kadie, "Explore Solvable and Unsolvable Equations with SymPy," in *Proceedings of the 24th Python in Science Conference*, in Proceedings of the Python in Science Conference. 2025, pp. 38–53. doi: 10.25080/kcht7476.

[25] S. S. Kumkov, S. Le Ménec, and V. S. Patsko, "Zero-Sum Pursuit-Evasion Differential Games with Many Objects: Survey of Publications," *Dynamic Games and Applications*, vol. 7, no. 4, pp. 609–633, 2017, doi: 10.1007/s13235-016-0209-z.

[26] M. Zargham, "ADCS lifecycle demo: requirements traceability with structural, symbolic, and numeric evidence records." [Online]. Available: https://dynamicalsystemsgroup.github.io/ADCS-lifecycle-demo/

[27] R. Mehta, "GDS-Axelrod: an interactive evolution-of-trust showcase built on the GDS compositional game-theory ecosystem." [Online]. Available: https://dynamicalsystemsgroup.github.io/gds-axelrod/

[28] M. Zargham and J. Shorish, "Generalized Dynamical Systems Part I: Foundations," Vienna, Austria, Working Paper, 2022. doi: 10.57938/e8d456ea-d975-4111-ac41-052ce73cb0cc.

[29] M. Zargham and I. Ben-Meir, "Protocols and Institutions," in *Cryptoeconomic Theory (Advances in Science and Technology in the Age of AI)*, World Scientific, 2026, ch. 7, pp. 157–179. doi: 10.1142/9781800618664_0008.

[30] W3C OWL Working Group, "OWL 2 Web Ontology Language Document Overview (Second Edition)," W3C Recommendation, 2012. [Online]. Available: https://www.w3.org/TR/owl2-overview/

[31] M. Zargham, "Formalizing Document Assurance: A Topological Framework for Verification, Validation, and Human Accountability," in *Proceedings of the INCOSE International Symposium 2026*, 2026.